SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Project Name: HireFlow — AI-Powered HR Hiring Pipeline

Group Name: We googled everything

Email: umhackathon@um.edu.my

1. Introduction:

Traditional recruitment processes are manual, slow, and prone to human bias. HR teams spend excessive time on repetitive tasks like CV screening and scheduling, often missing top talent due to delays. Company Y identified a market need for an intelligent, automated pipeline that reduces time-to-hire while maintaining high evaluation standards through AI-conducted interviews.

1.1. Purpose:

The System Analysis Documentation (SAD) highlights the technical scope, design decisions, and system boundaries behind the development of HireFlow. It serves as a blueprint for the MVP's technical feasibility.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. HireFlow is a cloud-native platform. The services are separated into modular components deployed via Docker. The core consists of a React frontend and an Express.js backend that interact independently with a PostgreSQL database and the external ILMU AI (GLM) service.
2. **Data Flows:**
    a. Demonstrates how candidate data flows from the public application portal, moves through the backend API for AI parsing and scoring, gets stored persistently in PostgreSQL, and finally returns live status updates to the HR dashboard.
3. **Model Process:**
    a. The model shows the end-to-end workflow of the core features: submitting an application, performing automated CV screening, scheduling an AI-led interview, conducting the interview, and generating a ranked shortlist for HR.
4. **Role of Reference:**
    a. This document is a reference for all developers and testers to enable the overall high-level architecture of this project, ensuring clear API usage contracts and a well-documented codebase.

1.2. Background:

Manual recruitment is plagued by "resume rot" and human bias. HR teams often spend 60% of their time on initial screening rather than high-value interviews. HireFlow was born to solve this by automating the "low-level" screening and technical testing, allowing HR to focus solely on final cultural fit and hiring decisions.

1. Previous Version:

    Traditional recruitment processes are often manual, slow, and prone to human bias. HR teams spend excessive time on repetitive tasks like CV screening and scheduling, often missing top talent due to delays. Company Y identified a

market need for an intelligent, automated pipeline that reduces time-to-hire while maintaining high evaluation standards through AI-conducted interviews.

2. Changes in Major Architectural Components
    - Public Candidate Portal: For CV uploads and application tracking.
    - HR Dashboard: A premium management interface for job configuration and candidate shortlisting.
    - AI Interview Room: A proctored environment where an AI agent conducts real-time technical and behavioral assessments.
    - Core Reasoning Engine: Built on ILMU AI (GLM-5.1) for CV parsing, question generation, and candidate scoring.
3. New capabilities that are only introduced in this
    - Autonomous Technical Interviews: AI agents capable of evaluating DSA, MCQ, and behavioral responses.
    - Integrity Proctoring: Automated detection of tab-switching, camera loss, and copy-paste events.
    - Agentic Shortlisting: Dynamic ranking based on a weighted formula (CV + Interview Performance - Penalties).

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
HR Manager	Create job postings, set AI thresholds, review ranked shortlists.	Efficient pipeline management, reduced manual screening, high-quality candidates.
Candidate	CV submission, participation in AI-led technical interviews.	Seamless interview experience, clear instructions, and fair, automated evaluation.
Development Team	Build and maintain the platform and AI integration.	Modular codebase, robust API contracts, and scalable AI service integration.
QA Team	Validate system behavior, proctoring accuracy, and AI scoring logic.	Comprehensive test cases for "cheating" detection and edge-case interview responses.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Platform (HR Dashboard + Candidate Interview Room)
Architecture	Client-Server Architecture with Centralized AI Service Layer
Deployment	Docker-containerized environment (PostgreSQL, Node.js)

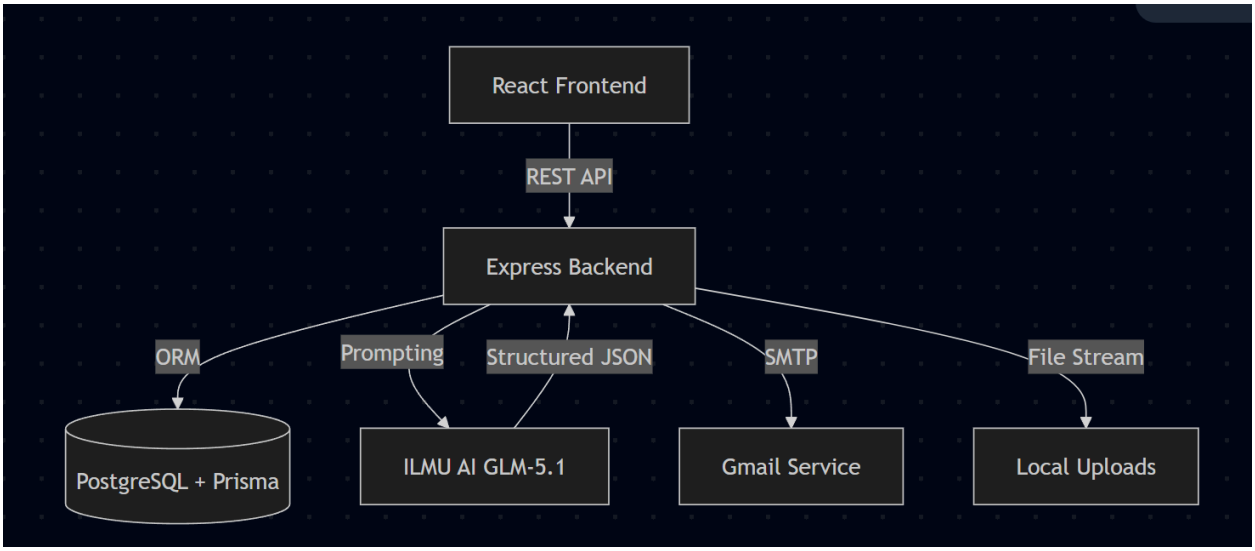
HireFlow is structured as a full-stack application. The React Frontend provides two distinct experiences: a premium dashboard for HR and a proctored, distraction-free environment for candidates. The Express Backend serves as the orchestrator, managing a PostgreSQL database via Prisma ORM and coordinating multi-step reasoning tasks with the ILMU AI GLM service.

2.1.2 LLM as Service Layer (GLM Integration)

The ILMU AI (GLM-5.1) is integrated as a central service layer via glm.service.ts. Unlike a generic AI module, it acts as the "brain" of the workflow:

- **Prompt Construction:** Prompts are dynamically built using candidate CV text, job requirements, and real-time interview answers.
- **Context Window Management:** CVs are parsed into structured JSON to fit within token limits. For interviews, only the current question and previous answer context are maintained to optimize performance.
- **Parsing:** The system enforces structured JSON output from GLM to ensure backend services can immediately process scores and reasoning.

2.1.3 Dependency Diagram



a. How the prompts are being constructed and sent to the GLM API

Prompts are dynamically constructed in glm.service.ts. For CV screening, the service extracts raw text from the CV file, concatenates it with the job requirements retrieved from the database, and wraps it in a strict JSON-enforced instruction template. This is sent via an HTTPS POST request to the GLM API.

b. What goes into the context window?

1. System Persona: Instructions defining the AI's role (e.g., "You are an expert HR analyst").
2. Context Data: Raw CV text, job descriptions, or candidate answers from the interview session.
3. Format Constraints: Explicit JSON schemas requested in the prompt to ensure deterministic parsing.

c. How is your system receiving, parsing the responses and passing to the next system component?

When the GLM API responds, the system extracts the content using regex to find the JSON payload (stripping markdown backticks). extractJsonFromResponse parses it into an object. If the parsing fails or the LLM is unavailable, the system falls back to a deterministic heuristic scoring algorithm. The parsed object is then passed to the workflow engine to trigger the next state (e.g., automatically moving a candidate to "AI_INTERVIEW_INVITED" if the score exceeds the threshold).

d. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM

Before reaching the GLM, file inputs (like long PDFs) are parsed into plain text. If

the text exceeds the model's token limits, it is truncated or chunked (prioritizing experience, skills, and education sections). The system ensures prompts stay within the optimal context window to prevent truncation by the API, enforcing a maximum character count before the HTTPS request is dispatched.

e. Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]

1. Candidate Portal to Backend (Application & Interview):
   - POST /api/v1/candidates/apply: Submits the candidate's CV for parsing and triggers the GLM service.
   - GET /api/v1/interviews/session/:token: Retrieves the AI interview session data.
   - POST /api/v1/interviews/session/:token/answers: Submits answers iteratively during the AI interview.
   - POST /api/v1/interviews/session/:token/proctor-events: Logs integrity events to the backend.
   - POST /api/v1/interviews/session/:token/submit: Completes the interview and triggers the final GLM evaluation.

2. HR Dashboard to Backend (Management):
   - GET /api/v1/dashboard: Fetches hiring funnel statistics and job metrics.
   - POST /api/v1/jobs & PATCH /api/v1/jobs/:id/prescreen-config: Job creation and configuring AI auto-screen thresholds.
   - POST /api/v1/candidates/:id/actions/accept-cv (and similar workflow actions): Triggers state transitions in the backend state machine.
   - GET /api/v1/interviews/ranked-shortlist: Fetches the AI-evaluated candidate ranking.

3. Backend to External Services:
   - AI Service: Express API (glm.service.ts) calls POST https://api.ilmu.ai/v1/chat/completions to retrieve reasoned output from the ILMU AI model.
   - Database: Express communicates with PostgreSQL to ensure persistence of candidate data and state history.
   - Email: The workflow-automation service communicates with Google SMTP to dispatch unique tokens to candidates.

f. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data

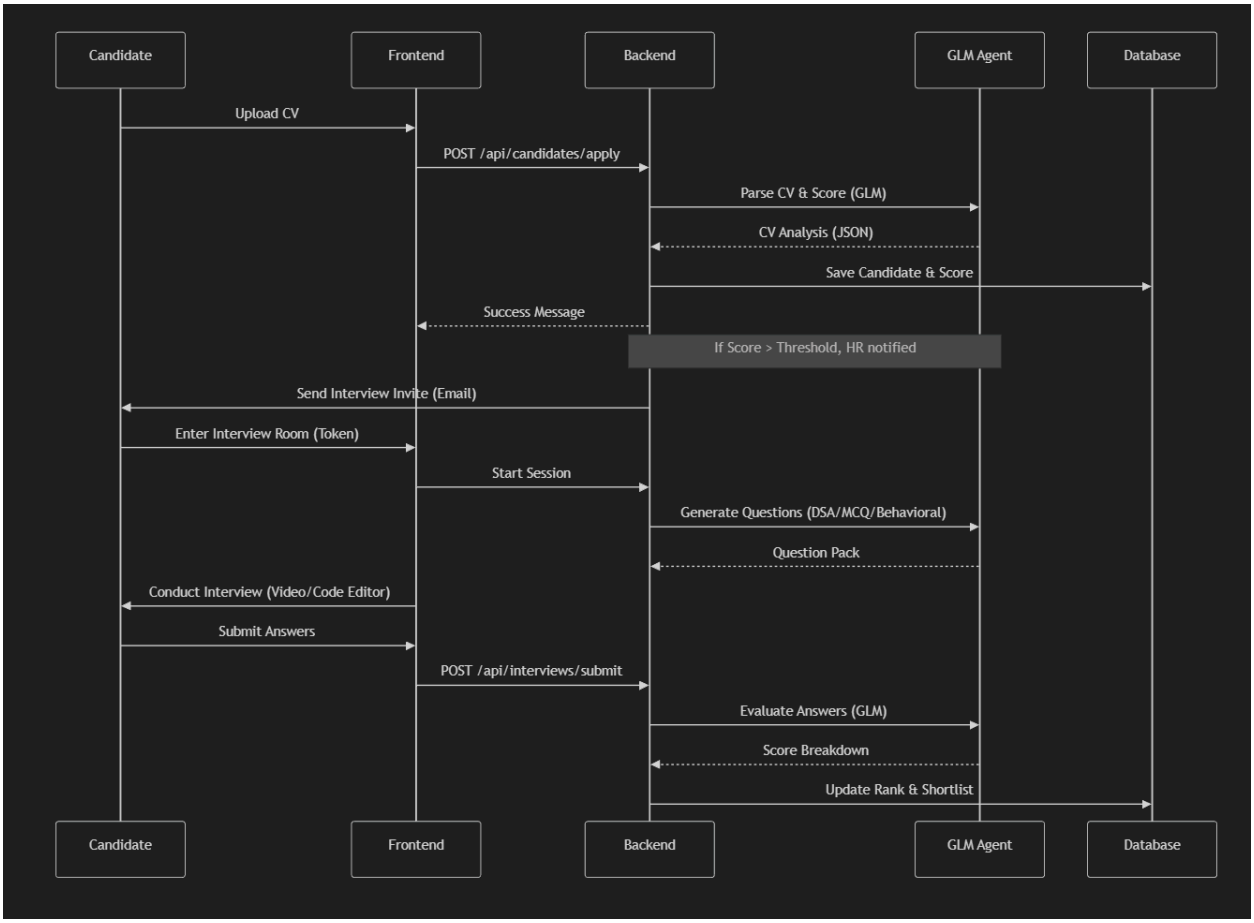
1. Frontend to Backend: The React Frontend calls Express API endpoints (e.g., POST /api/v1/candidates/apply for CV uploads). Live polling enables real-time updates for workflow state changes.
2. Backend to Database: Express communicates with PostgreSQL via Prisma ORM to ensure persistence of job, candidate, and session data.
3. Backend to External API: The glm.service.ts communicates with the external ILMU AI GLM-5.1 API for reasoning tasks.

g. Must highlights the distributed components if the system is planning to be deployed across multiple services

Joinly AI Interviewer
In addition to the core web-based AI Interview Room, the architecture includes the Joinly AI Interviewer. Developed but not fully integrated into the main pipeline due to time constraints, this component is a Python-based AI bot designed to automate live interviews.

- Functionality: The bot autonomously joins Google Meet sessions using Selenium/Playwright, bypassing authentication walls.
- Interaction: Once in the meeting room, it acts as a virtual interviewer, asking the candidate technical and behavioral questions verbally, listening to the candidate's audio responses, and transmitting those responses back to the GLM for real-time evaluation.

2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend: React 18, Vite, Tailwind CSS (Modern, responsive HR Dashboard).
2.2.2. Backend: Node.js, Express, TypeScript (Type-safe business logic).
2.2.3. Database: PostgreSQL (Relational data for candidates, jobs, and audit logs).
2.2.4. DevOps: Docker

2.3. Key Data Flows

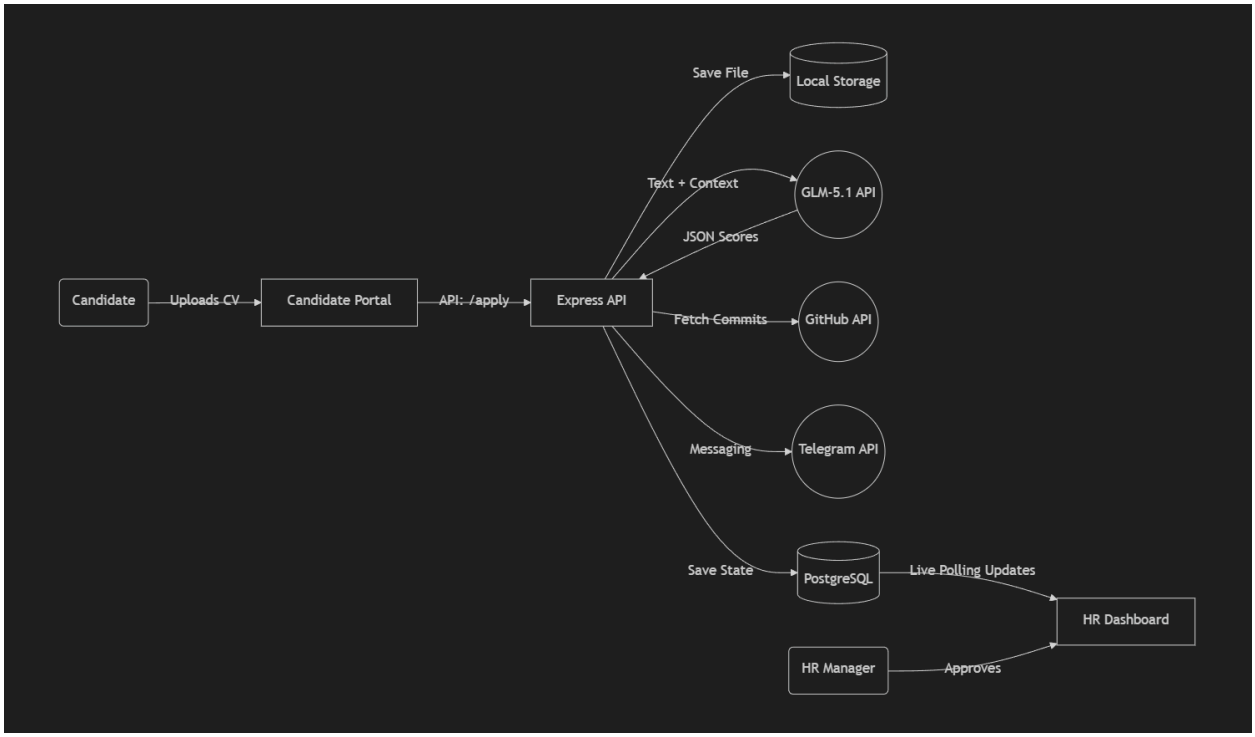
2.3.1. Here, do show the how data are moving through the system and how its structure and stored

Data moves through the system asynchronously to avoid blocking the user experience.

1. CV Upload: Candidate uploads CV (PDF/Word).
2. Parsing: Backend extracts text, saves to storage, and sends to GLM.
3. Evaluation: GLM returns a structured JSON evaluation.
4. Storage: Evaluation is saved to PostgreSQL, linked to the Candidate ID.

2.3.1.1. Data Flow Diagram (DFD)

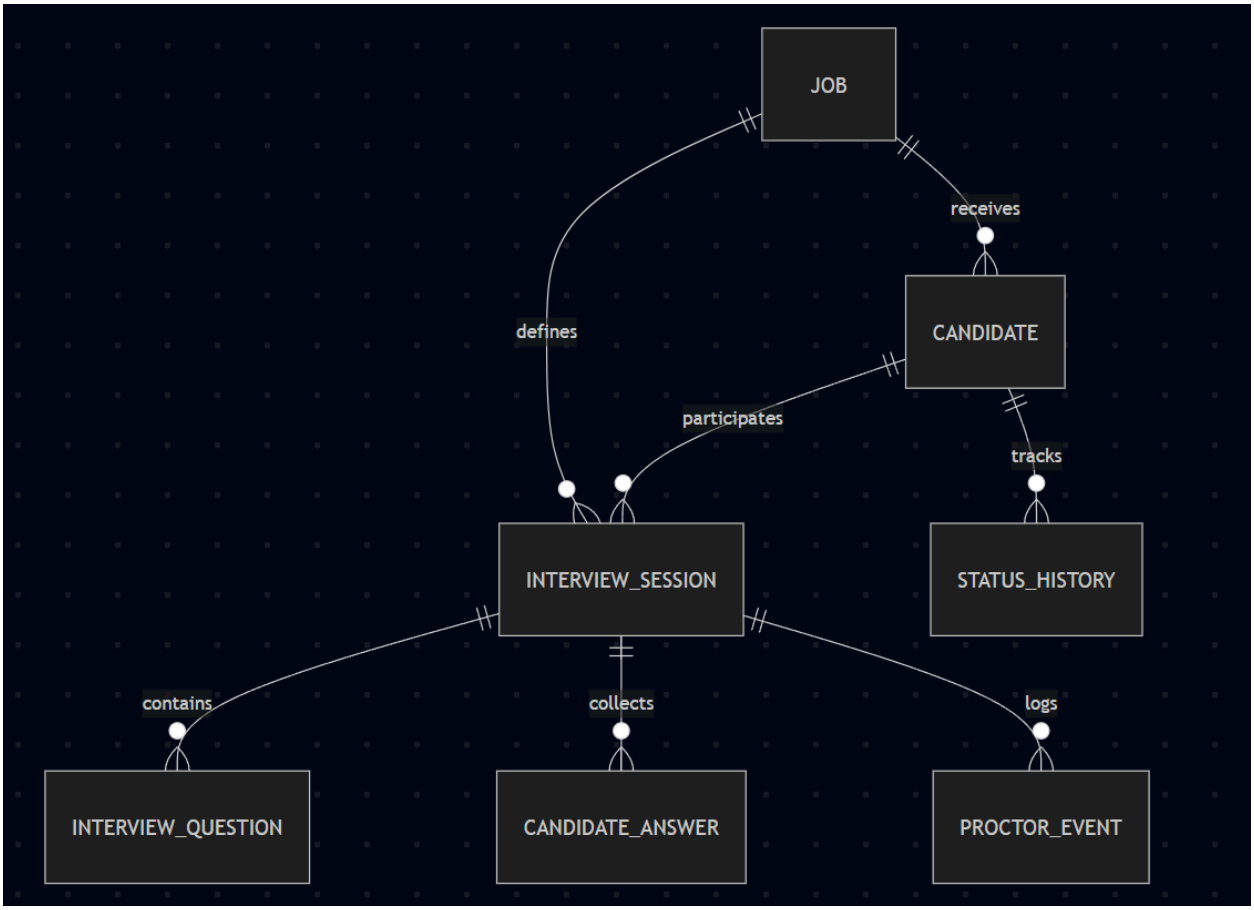
a. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases



b. *Show how proper laws are met (e.g. Black Hole)*

Legal and Compliance Standards (e.g., Avoiding Data "Black Holes"): To meet data processing standards and laws (like PDPA/GDPR principles), the system avoids "data black holes" by implementing an immutable StatusHistory table. Every state transition (e.g., APPLIED -> CV_UNDER_REVIEW) is logged with timestamps, trigger events, and metadata. This ensures full auditability and transparency of AI decisions.

2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1. Core features

#	Features	Description
1	**AI CV Auto-Screening**	Automatically parses PDFs, scores candidates against job requirements, and sets PASS/REJECT status.
2	**AI-Led Technical Interview**	A real-time room where GLM generates and evaluates DSA, MCQ, and Behavioral questions.
3	**Automated Proctoring**	Detects tab-switching and camera loss, logging events as "Integrity Penalties" in the final score.

4	**Ranked Shortlist**	Aggregates CV scores and Interview performance into a weighted leaderboard for HR.
5	**Offer Generation**	GLM analyzes the entire candidate journey to draft a personalized offer letter.
6	**Workflow Automation & UI**	Live polling for frontend state updates, visual workflow stage diagrams, and integrated state machines.
7	**AI Candidate Outreach via Telegram**	Proactive chat features where the AI agent contacts candidates on Telegram to clarify resume gaps.
8	**GitHub Profile Crawler**	Automated assessment of a candidate's code quality and commits directly from linked repositories.
9	**Bias Audit Module**	A dedicated reporting module to flag statistically anomalous score distributions across demographic proxies.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Reliability	Interview progress must not be lost on refresh.	LocalStorage sync + Heartbeat API to save draft answers in PostgreSQL.
Integrity	Detection of copy-paste and tab-switching.	JavaScript Event Listeners (blur/focus) triggered in the browser and logged to DB.
Token Latency	GLM evaluation should return within 5 seconds.	Asynchronous processing; UI shows a "Reasoning..." skeleton state.
Cost Efficiency	Prevent token "explosions" in long interviews.	Truncating CV text to relevant skills and limiting technical answers to 1000 characters.

3.3. Out of Scope / Future Enhancements

The following high-level features are planned for the "Grand Vision" of the product but are excluded from the UMHackathon 2026 scope:

a. Job Application LinkedIn Post Auto-Generation: Utilizing an AI Agent to automatically draft and publish job application announcements on LinkedIn.

b. Automated Onboarding Workflow: Automating the internal onboarding process, such as sending IT requests/tickets to schedule meeting rooms, and provisioning onboarding equipment like laptops and ID badges.

c. Joinly AI Interviewer Integration: While the Joinly AI interviewer bot is currently developed as a standalone project, fully integrating its automated meeting capabilities directly into the core HireFlow system is planned for future iterations.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: New AI evaluation prompts and scoring weight formulas will be released to 5% of HR users initially. The system is monitored for anomalies in scoring distribution. Once validated, it gradually increases to 100%. A/B testing is conducted to compare strict vs. lenient AI persona prompts to determine which yields a better candidate shortlist without bias.

4.1.2. **Emergency Rollback (ER) & Golden Release**: In terms of recovery, if the GLM API returns malformed JSON or parsing failure errors exceed 5%, ER is triggered. The automated pipeline will immediately fallback to the heuristic deterministic scoring method, and deployments will revert to the last stable "Golden Release" of the backend Docker image.

4.1.3. Priority Matrix:

The system uses the following Priority Matrix for incident management:

- P1 Critical: (e.g., GLM Service Down or Database Connection Failure) -> Action: Instant fallback to heuristic processing; alert Dev Team; if DB down, trigger Golden Release redeployment.
- P2 High: (e.g., Joinly Bot fails to bypass Google Meet auth) -> Action: System automatically sends candidates the fallback web-based Interview Room link instead of the Google Meet link.
- P3 Medium: (e.g., Proctoring camera event loss) -> Action: Log error, allow interview to continue without penalizing the candidate score for missing video feeds.

4.2. Monitoring:

4.2.1. Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.

Note: The following monitoring and alerting mechanisms outline the planned system health strategy for future production releases.

System health is planned to be monitored continuously:

- API Health: Endpoints are polled every 30 seconds. Response times > 2000ms trigger performance warnings.
- AI Token/Error Rates: Unsuccessful GLM API calls or rate-limit hits are logged. If failure rate > 5% in a 10-minute window, alerts are triggered.
- Frontend Live Polling: WebSocket/polling latency is tracked to ensure HR managers receive real-time updates without heavy server load.

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

a. Candidates have stable internet connections and functional webcams/microphones during the AI interview.
b. HR teams are using modern web browsers (Chrome, Edge, Firefox) to access the dashboard.
c. Uploaded CVs are in standard PDF or DOCX format containing selectable text (not scanned images requiring OCR).
d. Google Meet's authentication interface remains relatively consistent for the Joinly Bot to navigate.

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
ILMU AI API	Core Reasoning Engine for screening and interviewing.	**High:** Rate limits could halt interviews. Mitigation: Implementation of retry logic and caching.
PostgreSQL / Docker	Database persistence and containerized deployment.	Container crash leads to data unavailability; mitigated by persistent volume mounts.

Gmail SMTP Server	Sending unique interview tokens and candidate notifications.	Spam filters or rate limits could prevent invites from sending; mitigated by proper sender authentication.
Google Meet	Platform for the Joinly AI Interviewer meeting execution.	Changes to Meet's authentication UI could block the bot; mitigated by fallback web interview room.
Prisma ORM	Database schema management and migrations.	Schema mismatch during deployment; mitigated by strict migration version control.
Telegram Bot API	Direct outreach to candidates for clarifying resume gaps.	Rate limiting if too many messages are sent simultaneously; mitigated by message queuing.
GitHub API	Crawling public repositories to assess code quality.	API rate limits; mitigated by requiring authenticated requests and caching profile data.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. **Project Timeline:** Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]

Day 1 (17/4): Project brainstorm and planning
Day 2 (18/4): UI design, Prototype development
Day 3 (19/4): Prototype development
Day 4 (20/4): Backend schema integration & API structuring
Day 5 (21/4): Mentorship session, brainstorming 2

Day 6 (22/4): AI Interview implementation, Telegram hr chatbot
Day 7(23/4): AI Interview completion, CV investigation module, audit panel(prevent bias), Documentation Preparation, detail CV analysis implementation
Day 8 (24/4): Presentation Deck Preparation, Joinly AI Interview Development, UI Touch up,

5.2. **Team Members Role:** Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]
Yong Yew Sing - Project Manager/Team Leader - Project Management and monitoring, task distribution, progress tracking, front end developer
Yeah Jing Ze - Front end developer, UI Designer
Chew Zheng Wei - Backend Developer
Cheng Hao Wen - AI Workflow Engineer
Chek Chee Him - AI Workflow Engineer

5.3. Recommendations: If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]

To ensure future scalability, performance, and reliability, the following architectural enhancements are recommended:

- Redis Caching: Implement Redis Cache to store active interview sessions and high-traffic job configurations to reduce PostgreSQL load.
- Message Queues: Use a message broker (e.g., RabbitMQ or Kafka) for background processing of heavy LLM evaluations and CV parsing to prevent API timeouts during high concurrency.